

COMPUTACIÓN II – Práctica 1.

(1)- Escribir un programa para calcular la suma de los términos de la serie:

$$\sum_{i=1}^{\infty} \frac{1}{i^2} = \frac{\pi^2}{6}$$

- (a) Sumar los 10000 primeros términos en simple y doble precisión. Comparar los resultados al sumar los términos hacia delante y hacia atrás. (b) Repetir los cálculos anteriores con 10^6 términos. (c) Comentar los resultados.

```
#include <iostream>
#include <cmath>
using namespace std;
// Se hace el sumatorio en simple y doble precision
// Se suman primero 10000 y luego 1000000 terminos
// Ademas se realizan las sumas hacia delante y hacia atras ¿Por qué?
// Utilizamos moldes para asegurarnos de que i sea considerado como un numero
// real de simple o de doble precision en la funcion pow
int main()
{
    float S1A=0.0, S2A=0.0, S3A=0.0, S4A=0.0, PI=4.0*atan(1.0);
    double S1B=0.0, S2B=0.0, S3B=0.0, S4B=0.0, DPI=4.0*atan(1.0);
    const int n=2;
    // sumas hacia delante hasta 10000 terminos
    for (int i=1; i<=10000; i++) {
        S1A=S1A+(1.0/pow(float(i),n));
        S1B=S1B+(1.0/pow(double(i),n));
    }
    // sumas hacia delante hasta 1000000 terminos
    for (int i=1; i<=1000000; i++) {
        S2A=S2A+(1.0/pow(float(i),n));
        S2B=S2B+(1.0/pow(double(i),n));
    }
    // sumas hacia atras hasta 10000 terminos
    for (int i=10000; i>=1; i--) {
        S3A=S3A+(1.0/pow(float(i),n));
        S3B=S3B+(1.0/pow(double(i),n));
    }
    // sumas hacia atras hasta 1000000 terminos
    for (int i=1000000; i>=1; i--) {
        S4A=S4A+(1.0/pow(float(i),n));
        S4B=S4B+(1.0/pow(double(i),n));
    }

    cout << "SIMPLE PRECISION, 10000 TERMINOS" << endl;
    cout << "HACIA ADELANTE= " << S1A << " HACIA ATRAS= " << S3A << endl;
    cout << "SOLUCION EXACTA= " << pow(PI,n)/6.0 << endl;
    cout << endl;
    cout << "SIMPLE PRECISION, 1000000 TERMINOS" << endl;
    cout << "HACIA ADELANTE= " << S2A << " HACIA ATRAS= " << S4A << endl;
    cout << "SOLUCION EXACTA= " << pow(PI,n)/6.0 << endl;
    cout << endl;
    cout << "DOBLE PRECISION, 10000 TERMINOS" << endl;
    cout << "HACIA ADELANTE= " << S1B << " HACIA ATRAS= " << S3B << endl;
    cout << "SOLUCION EXACTA= " << pow(DPI,n)/6.0 << endl;
    cout << endl;
    cout << "DOBLE PRECISION, 1000000 TERMINOS" << endl;
    cout << "HACIA ADELANTE= " << S2B << " HACIA ATRAS= " << S4B << endl;
    cout << "SOLUCION EXACTA= " << pow(DPI,n)/6.0 << endl;
    cout << endl;
    return 0;
}
```

(2) - Calcular los números de Armstrong entre 0 y 999. Un número de Armstrong es un número tal que la suma de sus dígitos al cubo es igual al propio número. Por ejemplo el 371, ya que $371=3^3+7^3+1^3$.

```
#include <iostream>
using namespace std;
// el programa calcula los numeros de Armstrong entre 0 y 999. Un numero de
// Armstrong, es un numero tal que la suma de sus digitos al cubo es igual al
// propio numero. Por ejemplo el 371, ya que  $371=3*3*3+7*7*7+1*1*1$ 
int main()
{
    int abc, a3b3c3, contador=0;
    for (int a=0;a<=9; a++) {
        for (int b=0;b<=9; b++) {
            for (int c=0;c<=9; c++) {
                abc=a*100+b*10+c;
                a3b3c3=(a*a*a)+(b*b*b)+(c*c*c);
                if (abc==a3b3c3) {
                    contador=contador +1;
                    cout << "Armstrong number " << contador << ": " << abc << endl;
                }
            }
        }
    }
    return 0;
}
```

(3) - Hacer un programa para calcular el factorial de los 100 primeros números enteros.

(a) Usando variables enteras.

(b) Usando variables en simple precisión.

(c) Usando variables en doble precisión.

Comentar los resultados y reflexionar sobre las posibles causas que los justifiquen.

```
#include <iostream>
using namespace std;
int main()
{
    int fact1=1;
    float fact2=float(1.0);
    double fact3=double(1.0);

    for (int i=1; i<=100; i++) {
        fact1=fact1*(i);
        fact2=fact2*float(i);
        fact3=fact3*double(i);
        cout << i << "      " << fact1 << "      " << fact2;
        cout << "      " << fact3 << endl;
    }
    return 0;
}
```

```
// Otra posible solución utilizando arrays
#include <iostream>
using namespace std;
int main()
{
    int fact1[101];
    float fact2[101];
    double fact3[101];
    fact1[0]=1;
    fact2[0]=float(1.0);
    fact3[0]=double(1.0);

    for (int i=1; i<=100; i++) {
        fact1[i]=fact1[i-1]*(i);
        fact2[i]=fact2[i-1]*float(i);
        fact3[i]=fact3[i-1]*double(i);
        cout << i << "      " << fact1[i] << "      " << fact2[i];
        cout << "      " << fact3[i] << endl;
    }
    return 0;
}
```